



## Experiment – 5

**Name:** Probal Dhali

**UID:** 24BAI71013

**Branch:** CSE (AIML) CS221

**Section/Group:** 24AIT\_NTPP-3 [B]

**Semester:** 5<sup>th</sup>

**Date of Performance:** 13/08/2026

**Subject:** Full Stack - II

**Subject Code:** 24CSP-337

### EXPERIMENT – 5.1

#### 1. **AIM:**

To design and implement RESTful APIs using Spring Boot with proper validation, standardized responses, and scalable architecture.

#### 2. **OBJECTIVE:**

1. To understand REST API design principles.
2. To implement CRUD operations using Spring Boot.
3. To maintain consistent request-response structures.
4. To apply Bean Validation to incoming data.
5. To configure secure communication between frontend and backend using CORS.

#### 3. **SOFTWARE REQUIREMENTS:**

1. Java JDK 17 or above
2. Spring Boot
3. IntelliJ IDEA / Eclipse / VS Code
4. Postman or any API testing tool
5. Maven or Gradle

#### 4. **THEORY**

**REST (Representational State Transfer)** is an architectural style used to build scalable and loosely coupled web services. In REST, data is treated as resources and accessed using HTTP methods such as **GET, POST, PUT, and DELETE**.

Spring Boot supports a layered architecture:

- **Controller Layer** – Handles HTTP requests.
- **Service Layer** – Contains business logic.
- **Repository Layer** – Handles database operations

A good REST API should be stateless, use consistent URLs, and provide standardized responses.

**Bean Validation** is used to validate incoming data using annotations such as `@NotNull`, `@NotBlank`, and `@Size`. This prevents invalid data from entering the application.

**CORS (Cross-Origin Resource Sharing)** allows a frontend application running on a different origin to communicate with the backend securely.

Standardized responses make APIs easier to understand and integrate with frontend applications.

## 5. IMPLEMENTATION/PROCEDURE

- Create a Spring Boot project using Java 17 or above.
- Configure required dependencies such as Spring Web, Validation, and JPA.
- Create the Controller, Service, and Repository layers.
- Create the required entity/model for posts and scheduling.
- Implement CRUD APIs using GET, POST, PUT, and DELETE.
- Apply Bean Validation to request fields.
- Create a standard response structure for successful and failed requests.
- Configure CORS to allow frontend-backend communication.
- Run the Spring Boot application.
- Test all API endpoints using Postman.

## 6. CODE

Student.java

```
package com.example.demo.model;
```

```
import jakarta.persistence.*;  
import jakarta.validation.constraints.Email;  
import jakarta.validation.constraints.NotBlank;  
import jakarta.validation.constraints.Size;  
import lombok.AllArgsConstructor;  
import lombok.Data;  
import lombok.NoArgsConstructor;
```

```
@Entity  
@Data  
@NoArgsConstructor  
@AllArgsConstructor  
public class Student {
```

```
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;
```

```
    @NotBlank(message = "Student name is mandatory")  
    @Size(min = 2, max = 50, message = "Name must be between 2 and 50 characters")  
    private String name;
```

```
    @NotBlank(message = "Email is mandatory")  
    @Email(message = "Email should be valid")  
    private String email;
```

```
    @NotBlank(message = "Department is mandatory")  
    private String department;
```

```
}
```

## index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Student Management</title>
<style>
body{font-family:Arial;background:linear-gradient(135deg,#667eea,#764ba2);padding:30px}
.box{max-width:700px;margin:auto;background:white;padding:25px;border-radius:20px}
input,button{padding:10px;margin:5px;border-radius:8px;border:1px solid #ddd}
button{background:#667eea;color:white;cursor:pointer}
.student{padding:15px;margin:10px 0;background:#f5f5f5;border-left:5px solid #667eea;border-radius:10px}
.del{background:#e74c3c}
</style>
</head>
<body>

<div class="box">
<h2>🎓 Student Management</h2>
<p id="count">0 Students</p>
<div id="list">No students yet.</div>
<hr>
<input id="name" placeholder="Name">
<input id="email" placeholder="Email">
<input id="dept" placeholder="Department">
<button onclick="add()">➕ Add</button>
</div>

<script>
const API="http://localhost:8080/api/students";

async function load(){
  let r=await fetch(API),j=await r.json();
  let list=document.getElementById("list");
  document.getElementById("count").innerText=(j.data?.length||0)+" Students";
  list.innerHTML=j.data?.length?j.data.map(s=>`
<div class="student">
<b>${s.name}</b><br>${s.email}<br>${s.department}
<button class="del" onclick="del(${s.id})">Delete</button>
</div>`).join(""): "No students yet.";
}

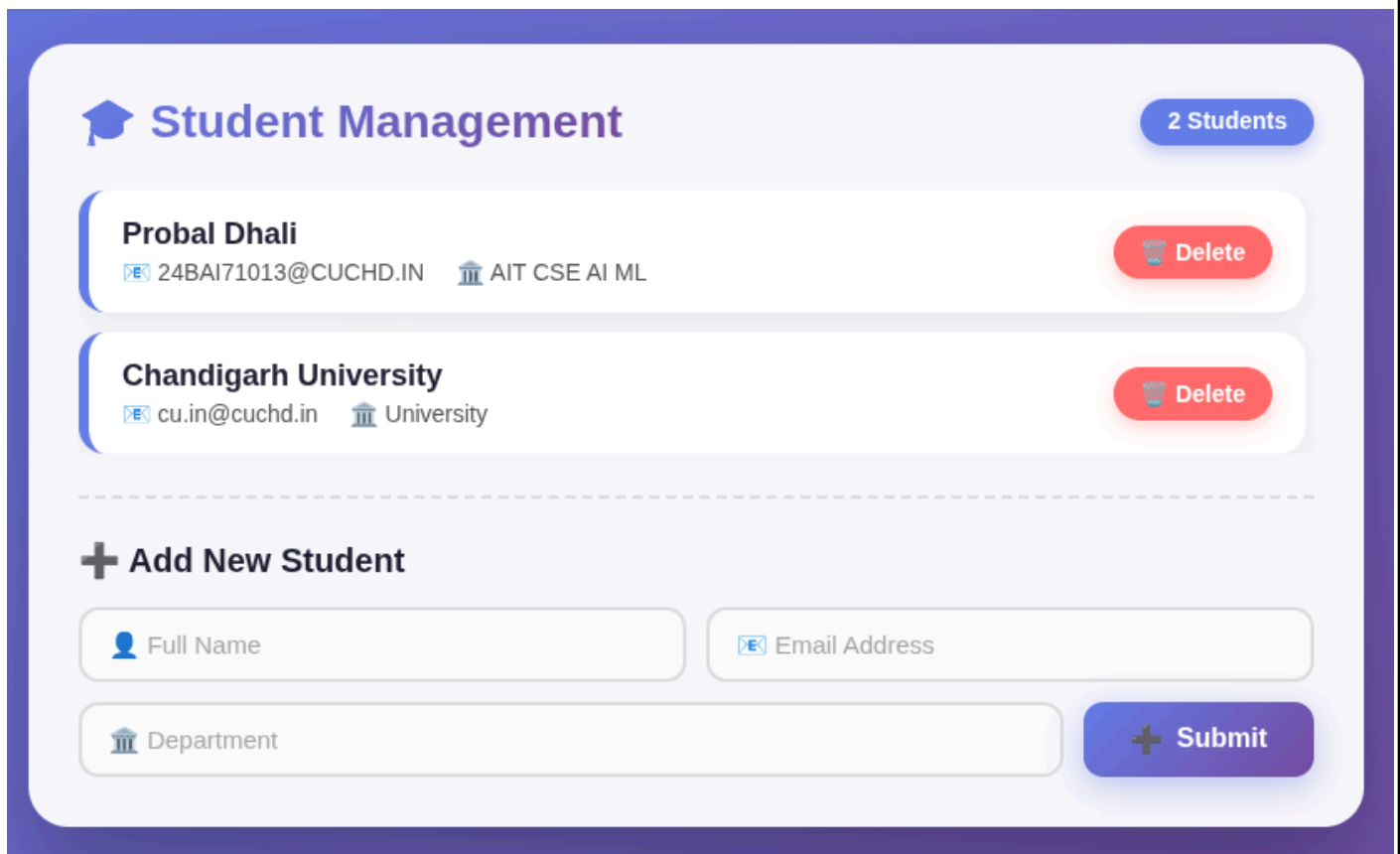
async function add(){
  let name=document.getElementById("name").value,
  email=document.getElementById("email").value,
  department=document.getElementById("dept").value;
  if(!name||!email||!department)return alert("Fill all fields");
  await fetch(API,{method:"POST",headers:{"Content-Type":"application/json"},
  body:JSON.stringify({name,email,department})});
}
```

```
document.querySelectorAll("input").forEach(x=>x.value="");
load();
}
```

```
async function del(id){
  if(confirm("Delete student?")){
    await fetch(API+"/"+id,{method:"DELETE"});
    load();
  }
}
```

```
load();
</script>
</body>
</html>
```

## 7. OUTPUT



- Fig 5.1:REST API Student Management for Creating a Resource

## 8. LEARNING OUTCOME

- Understand RESTful API principles.
- Develop CRUD APIs using Spring Boot.
- Implement Controller, Service, and Repository layers.
- Apply Bean Validation to API requests.



# CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

- Create standardized API responses.
- Configure CORS for frontend-backend communication.
- Test REST APIs using Postman.
- Understand scalable and maintainable backend architecture.

## EXPERIMENT – 5.2

### 1. AIM:

To implement global exception handling and structured logging in a Spring Boot REST API for building a robust, observable, and easily debuggable backend system.

### 2. OBJECTIVE:

1. To handle exceptions centrally using `@ControllerAdvice`.
2. To implement logging for tracking API requests and responses.
3. To use correlation IDs for tracing requests.
4. To standardize error responses.
5. To improve debugging and system observability.

### 3. SOFTWARE REQUIREMENTS:

1. Java JDK 17 or above
2. Spring Boot
3. IntelliJ IDEA / Eclipse / VS Code
4. Postman or any API testing tool
5. SLF4J and Logback
6. Maven or Gradle

### 4. THEORY

In large applications, handling exceptions separately in every controller creates code duplication and inconsistent error responses. Spring Boot provides global exception handling using `@ControllerAdvice`, which allows exceptions to be handled from a single centralized class.

`@ExceptionHandler` is used inside the global handler to process specific exceptions and return a standard response.

Logging is used to record important application events such as requests, responses, errors, and execution time. Spring Boot commonly uses SLF4J with Logback for logging.

Filters and Interceptors work as middleware between the client and controller. They can be used for request logging, authentication, request modification, and monitoring.

A correlation ID is a unique ID assigned to each request. Using MDC (Mapped Diagnostic Context), this ID can be added to log messages. It helps developers trace all logs related to a particular request.

Thus, global exception handling, structured logging, filters, and correlation IDs improve the reliability, maintainability, and observability of backend applications.

### 5. IMPLEMENTATION/PROCEDURE

- Create a Spring Boot REST API project using Java 17+.
- Create a REST controller with required API endpoints.
- Create custom exception classes for application-specific errors.
- Create a global exception handler using `@ControllerAdvice`.
- Use `@ExceptionHandler` to handle different exceptions.
- Create a standard error-response structure containing status, message, timestamp, path, and correlation ID.
- Implement a request logging filter.
- Generate a unique correlation ID using UUID.
- Store the correlation ID using MDC.
- Log request method, URL, response status, and execution time.

- Test both successful and erroneous API requests using Postman.
- Verify that all logs related to a request can be traced using its correlation ID.

## 6. CODE

### Student.java

```
package com.example.demo.model;
import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50, message = "Name must be 2-50 chars")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "Department is required")
    private String department;
}
```

### index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Student Dashboard</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
<style>
body{background:#eef2ff;font-family:Arial;padding:30px}.card{border:0;border-radius:20px;box-shadow:0 10px 30px #0002}.btn{border-radius:20px}.badge{font-size:14px}
</style>
</head>
```

```
<body>
<div class="container">
<div class="card p-4">
<div class="d-flex justify-content-between mb-4">
<h2><img alt="graduation cap icon" data-bbox="158 201 178 214"/> Student Dashboard</h2>
<button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#addModal">+ Add Student</button>
</div>

<div class="row mb-3">
<div class="col"><div class="card p-3"><h3 id="total">0</h3>Total Students</div></div>
<div class="col"><div class="card p-3"><h3 id="depts">0</h3>Departments</div></div>
</div>

<div class="d-flex gap-2 mb-3">
<input id="search" class="form-control" placeholder="Search..." onkeyup="filterData()">
<button class="btn btn-success" onclick="exportCSV()">Export</button>
</div>

<table class="table table-hover">
<thead><tr><th>#</th><th>Name</th><th>Email</th><th>Department</th><th>Action</th></tr></thead>
<tbody id="table"></tbody>
</table>
</div>
</div>

<div class="modal fade" id="addModal">
<div class="modal-dialog"><div class="modal-content p-3">
<h4>Add Student</h4>
<input id="name" class="form-control mb-2" placeholder="Name">
<input id="email" class="form-control mb-2" placeholder="Email">
<input id="dept" class="form-control mb-2" placeholder="Department">
<button class="btn btn-primary" onclick="addStudent()">Save</button>
</div></div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
<script>
const API='http://localhost:8080/api/students';
let students=[];

async function load(){
  let r=await fetch(API),j=await r.json();
  if(j.success){students=j.data||[];filterData()}
}
function filterData(){
  let q=document.getElementById('search').value.toLowerCase();
  let data=students.filter(s=>
    s.name.toLowerCase().includes(q)||
    s.email.toLowerCase().includes(q)||
    s.department.toLowerCase().includes(q));
  </script>
```



```
document.getElementById('table').innerHTML=data.map((s,i)=>`
<tr><td>${i+1}</td><td>${s.name}</td><td>${s.email}</td>
<td><span class="badge bg-primary">${s.department}</span></td>
<td><button class="btn btn-danger btn-sm" onclick="del(${s.id})">Delete</button></td></tr>`).join("");
document.getElementById('total').textContent=students.length;
document.getElementById('depts').textContent=new Set(students.map(s=>s.department)).size;
}
async function addStudent(){
let name=document.getElementById('name').value,
email=document.getElementById('email').value,
department=document.getElementById('dept').value;
if(!name||!email||!department)return alert('All fields required');
await fetch(API,{method:'POST',headers:{'Content-Type':'application/json'},
body:JSON.stringify({name,email,department})});
bootstrap.Modal.getInstance(document.getElementById('addModal')).hide();
load();
}
async function del(id){
if(confirm('Delete student?')){
await fetch(`${API}/${id}`, {method:'DELETE'});
load();
}
}
function exportCSV(){
let csv='Name,Email,Department\n'+students.map(s=>`${s.name}","${s.email}","${s.department}").join("\n");
let a=document.createElement('a');
a.href=URL.createObjectURL(new Blob([csv],{type:'text/csv'}));
a.download='students.csv';a.click();
}
load();
</script>
</body>
</html>
```

## 7. OUTPUT

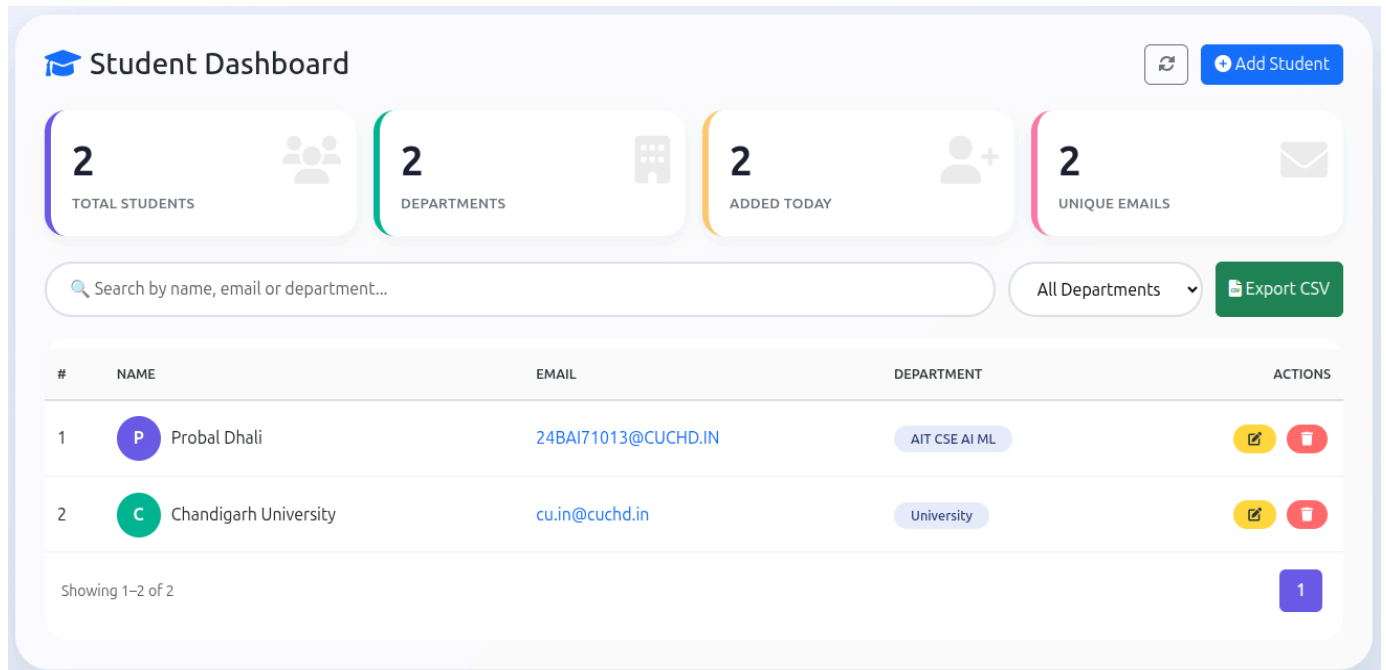


Fig 5.2: REST API Response

## 8. LEARNING OUTCOME

- Implement global exception handling using `@ControllerAdvice`.
- Create custom exceptions and standardized error responses.
- Implement request and response logging.
- Understand filters, interceptors, and MDC.
- Generate and use correlation IDs.
- Trace requests through application logs.
- Improve debugging, monitoring, and observability of Spring Boot applications.